The Publishing Accelerator Toolkit (PacTk)

Amy Neustein, Ph.D.
CEO and Founder
Linguistic Technology Systems
Fort Lee, NJ
917-817-2184
amy.neustein@verizon.net

A Collection of Software, Compiler, and Runtime Tools
for Managing and Visualizing Multimedia Data Sets

Overview

In contemporary publishing, the standard workflow includes **XML**-based formats such as **JATS** or from which are compiled reader-visible documents such as **PDF**s. These intermediate files are distinct from those that authors submit directly, and they can be computationally accessed via generic **XML**-oriented protocols such as (Simple **API** for **XML**), , and (scripting extension). Unfortunately, **XML** itself is not explicitly built for representing textual data, and examining **JATS** or files via generic **XML** technology renders certain query and editing tasks difficult or impossible to implement. The **PDF** format is similarly opaque because internal **PDF** representations of document text include many anomalies due to ligatures, hypenation, font specs, and other layout-related discrepancies between **XML** inputs and user-visible output.

Given the limitataions of **XML** for natural-language documents, numerous competing formats have been developed that provide alternatives or extensions to **XML** structures, including **TEI** (Text Encoding Initiative), (Text-as-Graph Markup Language), **BIOC** (a biomedical text mining standard defined by PubMed Central), and Research Object Bundles (a Semantic-Web based specification that convers both text and data sets). With a proliferation of different text-encoding standards, query and traversal protocols that are developed for one specific format (e.g., **JATS** and its derivatives) cannot be directly applied to others. For example, the **XML** parent-child element relationship is replaced in with two different containment styles (logical and extensional), such that **DOM** and event-based parsers targeted at the simpler **XML** hierarchy must be reimplemented for the system. From the perspective of , text-encoding evinces a spectrum of distinct *topomorphic regimes* where a proliferation of incompatible document structures prohibits the emergence of common query, traversal, and metadata formats.

One important variation amongst text-encoding formats is the level of granularity for most textual elements. In **JATS**, , and most **JSON** encodings, the fundamental discursive unit is the individual paragraph. Elements on smaller scales (such as sentences or block quotes) are notated haphazardly at best. LaTeX employs reasonable sentence-boundary heuristics (that can be overriden via macros when necessary), but sentence objects are not expressed in any systematic manner (e.g., via auxiliary files) except for intermediate spacing. In **TEI** and **BIOC**, individual sentences *can* be represented, but this is not mandatory (thus **s** and **sentence** tags are available but not guaranteed in any particular document). By contrast, is explicitly built around sentence-level units, specifically a "topomorphic sentence-level object model" (). When all regular text in a natural-language document is explicitly contextualized in a delineated sentence object, many editing, indexing, and user-interface tasks become feasible (as outlined in the next section).

In contrast to the other formats mentioned here, is not a single file format but rather a collection of code libaries and classes, with data types representing specific textual elements (sentences, paragraphs, **PDF** pages, annotations and comment-boxes, etc.). Query and traversal protocols modeled via , **DOM**, etc., become expressed by object methods and operators exposed to **C++** code, for example (is

thus “topomorphic” in the sense that navigation between objects is determined by structural properties of divergent document formats, that manifest in competing notions of individuation for individual structure-sites and their interrelationships). In short, the object system synthesizes distinct structuring protocols found in competing formats such as JATS, , and BioC, allowing publishers’ workflows to incldluing each of these formats (and others) without sacrificing compatibility. is particularly aimed at publishers who feel constrained by the limitations of JATS or but are reluctant to try alternatives (, BioC, etc.) due to minimal software-ecosystem support and the lack of standardized and DOM-style interfaces.

The remainder of this document will discuss how can streamline editing, indexing, and document-preparation tasks, as well as improve User Experience for human readers. Later sections will also address how can contribute to broader projects where text documents are published together with data sets and/or multimedia presentations.

Shapefiles and Layer Styles

In the case of GIS data, digital mapping software such as QGIS (to cite the most popular open-source application; ArcGIS is also widely used, but less amenable to third-party extensions) enable users to visualize GIS constructions — especially regions and point-sets (taking specific locations as *de facto* “points”) — via formats such as shapefiles (**.shp**). Attribute layers may then be superimposed employing formats such as **.sld** (Styled Layer Descriptor) to specify how visual cues should be rendered: in the case of mapping contaminants for hazard mitigation, the different pollutants tracked could be assigned distinct colors, and affected regions indicated via semi-transparent overlays. Specific locations might be marked with icons whose appearance distinguishes different hazard categories (chemical leaks, fire, air pollution, harm to local ecosystems, public health concerns), perhaps evincing different sizes in accord with risk-assessment levels. This kind of example typifies the overall strategy of GIS User Interface design: color, size, iconography, and related visual conventions allow digital maps to be employed as summarial overviews of GIS data sets. However, such formats as SLD do not support annotations for visual displays beyond the cartographic sketch; e.g., they do not proscribe how attribute data might be summarized in separate dedicated windows (like dialog boxes) or rendered as multimedia graphics.

In this context, the DaSemio tools are intended to enable developers’ formulation of data formats and protocols extending or replacing SLD (and its analogs in other domains), allowing multimedia interop to be explicitly regulated via data-set materials. In the case of environmental-hazard examples, SLD files governing GIS displays for contamination-related data sets could be augmented with stipulations for how chemical data would be visualized in software such as IQmol — that is, ensuring that users are presented with a consistent interface when loading molecular information into IQmol based on the antecedent digital map-locations. Similarly, multimedia applications such as video players could load annotations likewise stored in a data set; for instance, geotagging video frames would permit users to pause the video at a particular point and then, with interop enabled, have the map-display automatically scroll to the GIS location presently visible on the video screen. Video annotation might also superimpose text or graphics over segments of continuous frames, to more granularly relate the video content to the information tracked within the current data set.

For these scenarios, presentation formats such as SLD must be significantly broadened in scope in order to recognize annotations and cross-application messaging spanning diverse media types. Continuing the hypothetical environmental-hazards case study, a QGIS user could visualize GIS locations marked as contamination risks by loading data sets such as those archived under the Environmental Protection Agency’s “Toxics Release Inventory” (TRI) program. To make the display useful, one then needs to superimpose some form of base map (e.g., Open Street Map tiles) and construct a style definition to regulate how data-set contents are cartographically iconified. In principle, that could be achieved opening a style file, if one were available; often (as with TRI) that is not the case, and even if so a truly effective display might require additional fine-tuning beyond static style declarations.

The preferred QGIS approach in this case — which is typical of many science applications — is to automate and refine the data-set load process via Python scripts (or other embeddable languages, such as JavaScript). While supporting embeddable interpreters is important, choosing Python and other “general-purpose” scripting platforms introduces substantial added dependencies that complicate the software’s build process and end-user operations. From a developer’s perspective, linking against platforms such as Python often demands juggling multiple installations and nontrivial build steps (these are not the sort of dependencies that can be encapsulated with a single **cmake** install-prefix line). From an end-user’s perspective, the host software has to coordinate with an external package manager insofar as scripts have their own dependencies, and it is frequently impossible to resolve coding errors or versioning conflicts within the application itself.

These are the sorts of problems we propose to address via embedded Virtual Machines (as outlined in the next section), which enable a fully-operational scripting platform to be embedded in applications without any external dependencies at all, simply by adding a suite of source-code files to the host-application project.

Lacunae

In sum, we have reviewed a handful of technical challenges associated with multimedia data sets. As a rule, current applications and data formats have not been implemented with multimedia interop as a important feature, so very few software or code libraries have been

designed with relevant capabilities in mind. This is why we believe a package of extension and plugin libraries is necessary, to instantiate the sort of cross-application ecosystem allowing users to access and reuse multimedia data sets in a consistent and intuitive manner.

So as to build up that ecosystem, **DaSemio** introduces components largely organized into three groups: compiler tools, multimedia cross-referencing, and ingredients for modular software aggregation (particularly **GUI**s for individual multimedia content-types; also libraries involving database architecture and peer-to-peer cloud/native hosting). These are outlined respectively in the following sections.

Compiler and Virtual Machine (VM) Components

Most **DaSemio** components address concerns related to data sharing/integration. Problems in this domain tend to fit into several categories: 1) data sets are typically shared in compacted formats that must be deserialized or decoded to become available for application users; 2) application and **GUI** state should evolve to reflect the information structure of individual data sets; and 3) in contemporary science, data sets are considered to be distinct publications that (ideally) are cited and annotated by analogy to text documents — not only referencing published data holistically but also granular parts or metadata contained therein. Anyone working in scientific publishing over the past dozen or so years will have noted that publishing houses, as well as organizations that fund/sponsor academic research, have in that period prioritized data transparency and research replication: authors are encouraged (or required) to provide access to data sets and experiment results in conjunction with research articles. Summarizing lab and/or data-acquisition protocols is also recommended, to facilitate others' double-checking research results and/or reproducing and generalizing/extending those results in alternate settings. Yet the technological infrastructure for data publication has not evolved in step with data-transparency paradigms. As a consequence, open-data repositories are populated with large numbers of weakly-curated resources which fail to meet standards such as **FAIR** sharing (Findable, Accessible, Interoperable, Reusable), insofar as "raw data" is not intrinsically accessible, reusable, or interoperable.

Exacerbating this problem, scientists and publishers often lack a technical understanding of how data sets are loaded and processed, which results in confusion about how **FAIR** protocols should be implemented in practice. For instance, it is not uncommon for authors to conflate data sets themselves with summarial or analytic presentations, such as charts/plots or statistical breakdowns. While there is nothing wrong with diagramming data-set statistics via inline tables or figures, open-data initiatives stipulate that data sets in their entirety should *also* be published as separate entities. When it comes to structuring complete data sets that *are* published as integral wholes, another tendency is to assume that data publications become "reusable" insofar as they are assembled in compliance with popular standards, such as domain-specific protocols enforced by **RDF** Ontologies or **XML** Document-Type Declarations. In reality, those forms of structural validators may guarantee that traversal algorithms proceed (and terminate) correctly, but such algorithms must still be implemented. This is "Problem 1" listed above: users' access to data sets through science-oriented software entails reading individual values/records by decoding the data sets' elements into application-level datatype instances; here, Ontologies are useful but not sufficient for reconstituting data sets in live memory. This problem is exacerbated in the context of specialized formats associated with specific scientific disciplines — for instance, **CDF** (Common Data Format) and **FITS** (Flexible Image Transport System) in astronomy and Earth sciences — which require dedicated parsers and/or binary stream consumers. Even with generic formats such as **XML** or **JSON**, specialized computer code is still necessary in order to visit document nodes individually, reading each atomic value for assembly into aggregate structures. In short, data sets must be "loaded" into research software in order to examine or manipulate them, which can only happen if software libraries are employed (or implemented) to parse and/or navigate around published information, as converted to traversable structures.

Since it is impractical for every science application to build parsers *de novo*, organizations often spearhead the deployment and standardization of code libraries specifically associated with individual file types; **NASA**'s repositories for **FITS** media are a good example (in some cases, such as **CDF** or **HDF5** — the current "Hierarchical Data Format" — standardized libraries are the *only* way to access resources; elsewhere, re-implementing deserialization algorithms is possible in theory but infeasible in practice). To preserve **FAIR** standards, such libraries should be actively maintained, ported to multiple Operating Systems, and packaged for a wide variety of computing environments, particularly ones most associated with scientific research (Python, **R**, **Qt**, Java, etc.). Yet publishers and research institutions often fail to coordinate the continued development and maintenance of data-sharing resources. For example, publishers often list "Supplemental Materials" or "Data Availability" links alongside article abstracts, but they do not track the availability of software components equipped to read the specific formats adopted by those accessory data sets. Also, a familiar complaint with general-purpose formats such as **NetCDF** and **HDF5** is that similar data sets are often encoded differently, so that application code (even when built on top of deserialization libraries) has to be rewritten for multiple (otherwise compatible) information sources; midstream "adapter" code is often not publicly accessible. Nor are text formats such as **PDF** or **ePub** joined to (de)serialization libraries so that scientific papers could be cross-referenced with data sets and multimedia content on a granular level, based on annotations keyed to individual technical terms/acronyms, sentences, or paragraphs, rather than at the coarser scale of entire documents (Problem 3 above). Similar comments could be made with respect to government agencies and other organizations charged with sharing civic, infrastructural (in the sense of Computer-Aided Design and Civil Engineering Informatics), or environmental data for public benefit. In effect, publishers as well as research and governmental communities have failed to appreciate the distinct technological challenges associated with open data sharing in civil and/or scientific contexts.

The Role of Compiler Technology in Data Sharing

In light of these difficulties, our solutions involve a dedicated Virtual Machine and Compiler Infrastructure specifically optimized for data sharing and modeling — in other words, the foundation of a technology stack designed intrinsically around open-data requirements, rather than trying to fit transparency initiatives into (for example) generic web-programming paradigms. To our knowledge, our work constitutes the only project which approaches the data-integration problem space from the perspective of compiler theory and VM engineering. The basic idea is to employ a largely self-contained VM component which could be dropped in to code projects (including in source-code fashion) with few external dependencies. Source libraries for parsing and navigating within domain-specific data formats can then be developed in code that compiles to this specific form of VM, and thereby made available to any software which loads or links against the VM as an embedded component or plugin. Updates to data network/deserialization libraries are henceforth handled by the VM in isolation from the host application. This approach significantly reduces the overhead of porting and maintaining data-sharing code: because the VM is largely autonomous vis-à-vis its host environment, any library updates propagate automatically across OS and computing platforms.

(We assume that projects are canonically implemented in C++, since most scientific software is written in C++ and most of the rest can load C++ libraries. The definitive metric for language "popularity" and influence is the TIOBE index, and the combination of C and C++, which is reasonable because most applications link against a mixture of both C and C++ libraries, represents over 20% percent of all existing computer code — far above all other languages apart from Python at 15% — and collectively C++, C, and Python measure equally to all other top-20 languages combined; if we consider that Python is typically used as "glue" code delegating to native C/C++ libraries behind the scenes, it should be clear that the foundation of scientific computing is still, as much as ever, native-compiled components whose source languages are C and C++ in some combination).

Embeddable VMs provide a flexible infrastructure for coordinating simulations via automated workflows; an intermediary for database queries (via special instructions and/or foreign-function interface calls to emit a particular database engines' native query language); a fall-back for binary data exchange when one or both endpoints do not link against (and can not dynamically load) compatible libraries/plugins to read/write the desired format; or a scripting basis for data-acquisition tasks. In the latter case, applications could use scripts against data-provider APIs in lieu of users manually visiting web portals, which can be very cumbersone and involve multiple separate steps. As another use case, consider Reference Implementations for specs which rely on constructions that are not natively supported by most programming languages (design-by-contract, measurement-unit annotations, certain forms of inter-type relationships, etc.). Even languages primarily designed for compiling to native code could benefit from a VM environment for prototyping grammar and semantics, most forms of testing (at least those not affected by variations in execution speed), and as a fallback option when source code does not build on a particular computer (due to, for instance, outdated system libraries; **glib**, say). VMs can also bridge multiple components for interop within foreign-function interface scenarios that must resolve such details as C++ name mangling. Popular languages for science-related-code, like Python and R, by contrast, often present significant difficulties when it comes to integration with native-compiled code libraries — especially components such as GUIs that employ frequent cross-module message passing. An encouraging development is the emergence of new languages (e.g., C++ dialects like cppfront, Carbon, jank, Zig, and Circle) that directly generate and link against native libraries, while also introducing advanced design-by-contract and memory-management capabilities (among other innovations) — although a richer compiler infrastructure could help these divergent projects cohere into a single C++ successor/extension framework (even if it has multiple top-level languages with different flavors, rather like the JVM); we can imagine, for instance, the Stroustrup/dos Reis Internal Program Representation (IPR) approach extending to C++ "successor" dialects. Here, too, embedded VMs are potentially valuable because they would permit variegated front languages to produce shared bytecode, without the overhead of a framework like LLVM which, once it becomes a forced dependency, may significantly complicate an application's build process.

Another rationale for VM-based implementations (even for languages that compile to native code) is to clarify compile-vs-runtime and template metaprogramming (TMP) details. Insofar as VMs and the compilers that generate their code may both be seamlessly embedded in applications, it is possible to expose compilation "hooks" able to call arbitrary C++ functions (for example) and manipulate VM bytecode directly — a level of transparency that is not feasible when generating native machine code instead. In several books we have discussed certain kinds of "back-door" communication between caller and callee procedures in terms of "channels," which when properly utilized can enable many new runtime and static-analysis features (we also submitted a brief demo on the "C++ insights" website: https://cppinsights.io/s/512c2700). There is some precedent for such techniques in existing C++ conventions, such as copy-elision and "Return Value Optimization" (RVO) through which (in effect) procedures share (partial) access to their caller's stack frame. One motivation for our own VM implementation is to model augmented RVO as a solution to existing C++ memory-management complications, related to extending temporary value lifetimes (proposed "lifetime annotations" import ideas from Rust into C++, but we propose a more flexible method to achieve similar safety guarantees — there's a brief summary on cppinsights at https://cppinsights.io/s/cac923c9). The main point here is that RVO represents one example of optimizations which can be modeled via what we call "channel algebra." It would be useful to free up such constructions so that compilers could extend VM code with other channel-based features in a relatively open-ended manner. Obviously, translating such models over to C++ compilers proper would require careful standardization (RVO is supported only in a systematically enumerated set of circumstances, for example), but this is consistent with the general theme of prototyping language extensions in a VM context and then investigating how and whether they are implementable for a "core" language in a compiler-neutral and

cross-platform manner.

A Compiler/VM infrastructure dedicated to data modeling and sharing has additional benefits, because it permits introducing features specifically oriented toward issues of data integrity and FAIR protocols. For example, specialized VM instructions could be provided for dimensional analysis, measurement units, range limits, shape constraints, "axis-labeled" types (e.g., distinguishing number-pairs whose coordinates are **x/y** versus **row/column**, **width/height**, **first/second**, etc.), and other metadata details addressing the semantic integrity — above and beyond syntactic propriety — of individual fields/records. Apart from validation, such semantic constructs serve as documentation and pedagogical tools for the scientific models and theories illustrated via data publications. Also, VMs provide a framework for scientific applications to expose their capabilities and user-visible state adaptably as data sets are loaded (Problem 2 above): individual data sets will often have information profiles that require special GUI formulations or analytic processing within the host application. Because data sets and digital models are not just static assertions of collected data points, but rather should function in effect as code libraries — with dynamic simulation and/or project-specific interaction and visualization features — models/sets should be engineered in environments which facilitate interop with scientific computing and advanced UI libraries. For instance, many image formats incorporate embedded metadata (FITS data units, TIFF tags, DICOM attributes, etc.), so applications require — in addition to the obvious graphics-display features — some mechanism to show users content from metadata fields, at least insofar as those parameters are intrinsic components of the information being shared (and not behind-the-scenes specs expected to remain invisible, such as color-depth descriptions instructing software how to render image rasters). Embedded VMs allow applications to run scripts published alongside data sets, which would dynamically build GUI windows or components, support domain-specific analyzers or workflows, and potentially expose certain parts of the overall application state (e.g., the toplevel window menubar; context menus; networking capabilities; or local filesystem access).

Interoperability across different scientific domains is limited because each discipline tends to have its own information-sharing ecosystem, often with decades-old roots, reflecting successive generations' favored theories about optimal data storage and processing. Formats emerging during the heydey of 90s-era Object Databases often reflect certain modeling assumptions, that were to a nontrivial extent displaced by the Semantic Web fashion of the 2000s, only to see "Ontology" schemas decline in popularity against 2010s analytics and machine learning. New tools and paradigms progressively complicate the goal of data interoperability, leading to new standards (often backwards-incompatible) which require new code libraries (cf., HDF5 versus HDF4) and even a degree of convergence between formerly incompatible formats (cf., HDF5 versus NETCDF). These evolutionary trends present opportunities for data-management libraries to adapt and widen in scope, but institutions have to take the corresponding initiative. Advances in data formats often correlate with innovations in software engineering and tooling, so larger technological factors can potentially be a catalyst for institutional re-prioritization of data-sharing capabilities. For example, the Biden Administration recently (on February 26, 2024) released a "Statements of Support for Software Measurability and Memory Safety" that has garnered a lot of commentary in the computer science community; and may portend a (further) entrenchment of concepts such as lifetime annotations and software metrics among developers of programming-language toolsets. On technical grounds it would be appropriate for these concerns to be explicitly addressed by future versions of libraries supporting formats such as NETCDF and HDF (or their successors); see the above discussion about design-by-contract for concrete examples.

Constraint Declarations and Other VM Use-Cases

Meanwhile, at least some information formats imply or presuppose the ability to represent computational procedures as components *of* a data set (as opposed to tools for working *with* a data set). For example, general-purpose modeling languages such as EXPRESS sometimes support constraint declarations as intrinsic features of schemas, e.g., for defining derived types (wherein one type refines another via boolean predicates, restricting its domain). Constraint-expressions may, at least sometimes and in theory, be notated via data structures built from primitive true/false tests such as equality and greater/less; however, it is arguably more accurate to treat constraints as subroutines behaviorally modeled via a reasonably expressive procedural language, one whose semantics would be consistent with general-purpose programming languages. For instance, influential EXPRESS-based standards, such as Industry Foundation Classes (IFC) in engineering, require support for "procedural" data exchange (constituting "level 4" interoperability in the IFC context) to ensure, above and beyond raw data duplication, that multiple components accessing a particular data set are able to transform it in compatible ways.

A similar situation arises when formulating meta-modeling paradigms via graphs (or, more precisely, hypergraphs), insofar as representational paradigms diverge based on what forms of internal structuration is recognized for individual (hyper)nodes — i.e., a hypernode aggregates smaller information-units via potentially multifarious formations, including key/value property sets; single-dimensional tuples/arrays; one-dimensional arrays which internally contain (potentially) multiple sublists with their own insertion-points, as with fields in multi-value databases; multi-dimensional arrays/tensors; nested document structures, etc. Hypergraph formats may differ with respect to which forms of intra-node structuration should be supported as intrinsic *kinds* of hypernode and which should be modeled indirectly or aggregatively via relations *between* (hyper)nodes. For maximum semantic generality, arguably the best approach is to equip hypergraph models with a foundational hypernode structure (e.g., a one-dimensional array; we have published analyses advocating for a "cocyclic" model wherein one-dimensional arrays are formed from a fixed-size tuple concatenated with a resizable consistently-typed list, which introduces at least a provisional internal detailing for hypernodes) and a further capacity to construct "views" through which hypernodes could be queried according to more complex inner configurations (tensors, hierarchies, etc.) even if the "physical" encoding stores such

complexes indirectly (e.g., via multiple interlinked hypernodes). That form of semantics requires procedural capabilities to "adapt" underlying node (potentially multi-node) patterns to single-hypernode views (for instance, coordinate arithmetic to view a single-dimensional array as a two-dimensional matrix, or a three-dimensional cube, etc.). In database theory, a view *adapts* nodes/records so that they "respond" to query formulations whose structure might imply a different layout than natively supported by the database architecture; here again multi-dimensional arrays are a concrete example, because a query-expression could specify two or more dimensions' indexes whether or not the target site encodes arrays as collapsed to one dimension, or arrays of pointers, or sparse-matrix key/values, etc. Similarly, adapters can ensure that a single query format/language works against data sets with variegated storage mechanisms: the physical storage might be indexes into binary chunks, or atomic values from an underlying key/value store (**BerkeleyDB**, etc.), or "addressable units" of intrafile storage (as with **HDF**).

In short, there are numerous use-cases wherein evaluable subroutines get attached to specific sites or schemas within a data set, insofar as modeling contained information requires semantic details that can only be expressed procedurally, rather than via static accumulations of value-primitives. One then faces the question of how subroutines should be defined, because in the best-case scenario generic meta-models are independent of particular programming languages. Compiler/**VM** tools offer functionality to address this situation first by including, as part of the compiler workflow, at least one internal representation stage where subroutines' code may be queried and validated according to data-modeling specifications; and second by supporting annotations and triggers within bytecode for both static and dynamic analysis. Such capabilities help compensate for any idiosyncracies that are artifacts of procedures' particular implementation language.

Overall, then, a compiler/**VM** infrastructure serves data modeling at several levels: within the data set itself for semantic expressiveness; to streamline and standardize (de)serialization across all users of a data set; and to accommodate data sets' specific visualization and interactivity requirements — especially in terms of via host-application functionality and **GUI** components.

Cross-Referencing Across Diverse Multimedia Content Types

Our work on multimedia annotation and cross-referencing logically extends this kind of compiler/**VM** infrastructure, because data sets will sometimes cover a variety of distinct media formats that should be synchronized and integrated at the application level. As an example, consider software oriented toward domains such as **NASA**'s Earth Observing System, with functionality centered on satellite imaging and climate science. In this sort of context, consider a data set whose main components are **GIS** attribute layers and geotagged annotated videos. In order to render that content, a host application would need capabilities both to show **GIS** maps in a dedicated/interactive window with functions to overlay dynamically loaded geotagged data structures, and to show videos via a media player that could read and superimpose annotations on individual frames and frame-spans. While there exists *software* for both of those requirements (e.g., **ArcGIS**-style viewers and video production editors) there are not, in general, reusable cross-platform libraries that would allow such features to be packaged as modules dropped in to host applications — e.g., as separate application windows — by analogy to embedded "iframes" in a web application. In effect, native-compiled desktop software does not have an embedded object/iframe system allowing for modular component aggregation, to the same extent as such modular decomposition is possible in web programming.

To address this limitation, we have worked on code libraries whose purpose is to import functionality specific to distinct media types (**GIS**, video, image series, 360° photography, **3D** mesh objects, etc.) as mostly autonomous extensions to host applications. By standardizing such modules in a context focused on data interoperability and transparency, we can prioritize capabilities for managing shared data sets: multmedia viewers should not only render their correlated content-types, but dynamically adapt to the architecture and metadata conventions of data sets as they are loaded.

Cross-Referencing and Modularity

Cross-module integration is an inherent part of such adaptability, because semantic connections often exist between data layers attached to multiple media types. Geotagging video frames, for example, introduces a user pragmatics wherein **GIS** locations annotated for an individual video (or portion thereof) could be visited on a **GIS** map display, and vice-versa. For instance, videos documenting climate change in specific **GIS** locales might be cross-referenced with digital maps wherein visible overlays quantitatively reinforce the trends captured on film (for a concrete example, consider showing the effects of draught on a fresh water source, whose shrinkage over time gets rendered by superimposing prior water levels on current ones). Assuming an application has one window for digital maps and another for videos, the two windows should be able to work in sync, wherein (for example) a context menu activated on a paused video frame might include, as one action, an option to center the **GIS** display on the current frame's geotagged location (and, if applicable, load **GIS** attribute layers tied to the currently-visible video annotations).

A further dimension of cross-referencing is linking both data sets and multimedia resources with Natural Language texts, such as scientific publications — particularly with the kind of granular document annotations mentioned above, wherein specific words or phrases could be flagged for association with data-set and multimedia content. For example, text itself may be geotagged insofar as geospatial Named Entities — e.g., words or phrases which constitute the name of **GIS**-mappable locations (towns, buildings/infrastructure, rivers and other

waterways, toponymic features, archaeological sites, Superfund projects) — can be marked with latitude/longitude coordinates, allowing for multi-window application displays where document viewers are juxtaposed with interactive GIS windows. Similar principles apply to almost any technical discipline having Named Entities, as stipulated within Controlled Vocabularies, to designate objects or phenomena amenable to multimedia visualization (chemical compounds with three-dimensional molecular structures, proteins with folding potentials, solids with crystal morphology sometimes, etc.). DaSemio includes a novel file format which generates machine-readable text encoding as well as interactive PDF files, wherein annotations and cross-references are mapped to PDF page coordinates (in addition to text locations) so that users are able to see and interact with annotations on-screen alongside the underlying document. In particular, PDF pages may be converted to SVG files with annotations superimposed as a separate graphics layer, allowing for visible effects such as mouseovers, popups, and context menus, that are familiar to interactive SVG programming but usually available for PDF (and/or EPUB documents) in a limited and inflexible manner (e.g., PDF comment boxes).

Granular machine-readable text encoding has the additional benefit of enabling large-scale corpus analysis. We worked on our document-publishing code while preparing our book on *Innovative Data Integration and Conceptual Space Modeling for COVID, Cancer, and Cardiac Care* (Elsevier 2022), which reviewed a number of software projects in bioimaging and biomedical data mining. One of the case-studies for that analysis was CORD-19 ("**Covid19** Open Research Dataset"), a large repository of academic papers about Covid and Coronaviruses that was curated (early in the pandemic) by the Allen Institute for Artificial Intelligence ("AI2"). As AI2 openly acknowledged, CORD-19 suffered from inconsistent representation of both "ordinary" text and of special terms and character-sequences, such as chemical formulae (we gave one example of how a particular molecule was represented with subtle variations across multiple articles, so that the indexers which compiled machine representations parsed the formula in several different ways, obscuring the fact that — because all such representations designated the same entity — thematic links existed between the papers involved). Failure to properly encode cross-referenceable semantic overlaps between diverse artifacts dilutes the ability of text- and/or data- mining algorithms to identify interconnections and build semantic-network models of large-scale archives. Given their experience curating CORD-19, AI2 published a "call to action" highlighting the problems with existing text-encoding standards and encouraging publishers and research institutions to adopt more rigorous technology. For our Covid/Oncology book we implemented and utilized a publishing system that, we believe, addresses many of AI2's concerns, and we outlined this work over the course of several conference calls with Elsevier tech teams; but we are not aware, on a larger scale, of any systematic attempts within the publishing industry to acknowledge AI2's "call to action."

Anti-patterns in publishing technology go beyond text mining alone: microcitation and cross-referencing practices — especially when multiple genre and presentation/visualization media are involved — remain haphazard and often unstandardized. A comprehensive scientific and/or research corpus will likely encompass assets of diverse genres beyond just text documents: video, 3D, GIS, and so forth, as well as domain-specific data sets with their own media/visualization conventions. In many cases there is no standard system for cross-referencing resources from distinct media — even when there are obvious overlaps between content expressed in one media/genre and another, such as a chemical compound named (via molecular formulae) in a scientific text and also visualized in a three-dimensional model. Or, consider videos documenting the ecological effects of climate change, which could be cross-referenced against GIS maps (via latitude and longitude coordinates). In a cross-disciplinary context, many different forms of media and visuals can be relevant; **Covid-19** research for instance made extensive use of GIS models (tracking infectivity rates and related epidemiological data across the world) and also 3D models (of the virus's notorious "spike proteins" in particular), diagnostic images (such as "ground-glass opacity" in lung scans) and electron-microscope pictures of the virus itself. Despite the thematic correlation between these different media, however, the distinct software components which can show and analyze files and data sets of these various categories are often non-interoperable, as we emphasized above. Part of the problem is that data and file formats specific to individual media genres cannot necessarily be annotated with cross-references to other genres; but an equally significant limitation is that software is not equipped to identify and/or properly utilize such cross-references even when they are available.

Limitations within text corpora and multimedia data sets are interrelated, or at least stem from similar causes: in each case there is ambiguity with respect to the individuation and description of particular information/presentation units, limiting the extent to which disparate software components have a means to interoperate — in particular, to leverage shared data elements as a bridge from one component to another. This is an underlying issue when attempting to define microcitations mutually recognized by variegated multimedia technologies. Consider again the case of geotagging video frames: restricting GIS coordinates to a particular geographic area is one way to isolate specific parts of a GIS database (or any data set with GIS parameters as one data attribute, which might emerge from disciplines as diverse as epidemiology, sociodemographics, environmental science, ethology, anthropology, etc.). Likewise, referring to one segment within a video represents a potential form of microcitation (as opposed to the video as a whole). A microcitation protocol in these two domains would, by extension, allow application-level cross-references, so that software components dedicated to showing maps and videos, respectively, would at least have representations of the underlying data links permitting GIS locale to serve as a bridge between video and geospatial media. However, *absent* a common protocol the respective software components (digital map engines and video players) cannot interoperate. The analogous problem in the domain of text-annotations derives from inconsistent standards with respect to textual citations and references more fine-grained than traditional bibliographic and page citations.

Texture Annotations

Continuing the topic of multimedia cross-referencing and annotations, one of the more complex parts of this domain involves image series and picture markup, because of how image-processing workflows typically generate sophisticated feature vectors and color-space descriptions (this is especially true in contexts such as satellites or hyperspectral imaging, characteristic of astronomical or aeronautical technology). In particular, texture segmentation — in contrast to partitioning grayscale reductions via one-channel gradients (at zero-dimensional points) — depends on aggregate structures to characterize distinct textures and quantify textural "distance" such that cross-region boundaries might be identified. In other words, texture annotation has to document internally complex individual data structures. Moreover, texture-detection pipelines frequently rely on statistical "signatures" which — even if they accurately distinguish one region from another — do not specifically model the "semantic" or empirical characteristics of textures, seen from a perceptual or cognitive/conceptual perspective. Robust texture annotation therefore might demand image-analysis routines even beyond the primary segmentation, labeling, classification, or region-detecting workflow.

Empirically, textures can be described not only in terms of their statistical qualities, but via the physical and optical processes that cause color-regions to have a textured (rather than solid-color) appearance. Material phenomena for how pigments inhere in an object's surface typically have some variability of diffusion, so that a level of chromatic alternation is to be expected even within a region embodying "one" color in the sense of an underlying substance's intrinsic qualities (the green of a leaf, the blue of a body of water, and so forth). Of course, separate and apart from intrinsic color effects, optical details of light and shadow (plus glossiness, specularity, albedo, etc.) contribute their own articulations to texture geometry.

Optimally, as such, texture annotations would be equipped to characterize both the physical and optical processes which yield a specific *visible* (and photographable) texture, including, first, representations of pattern-symmetry; and, second, distinguishing the number of distinct "primary" colors that affect a given texture (insofar as many patterns are actually pigment and/or optical variability based on one single chromatic hue, whereas others interleave two or more primary colors). Standardizing "semantic" texture description is difficult, however, because the relevant image-analysis algorithms are supported very inconsistently. For example, one promising texture-segmentation approach is based on statistical topology — in effect, tracking how an image's topological properties evolve as it gets deformed via operations that tend to smooth out local color variations. Feature-vectors derived via these methods fall under the general mathematical heading of "Morse theory" and track "persistent homology groups" according to two axes which, in general, collectively measure the extent to which an image region should be deemed analytically important. Intuitively, such topological methods are particularly useful in conjunction with symmetry-detection, because pattern invariants are often measured most strongly at specific points in the persistent-homology spectrum (a level of topological simplification sufficient to cleanly suggest local-region centroids while preserving their homology-group persistence). Apart from the fact that identifying such Morse-spectrum points is necessary for extracting texture geometry most effectively, the topological details of where patterns are most precisely individuated (with respect to deformation metrics) represent one parameter for texture description. Unfortunately, however, although both symmetry-detecting and statistical-topology algorithms have been published, neither are a standard feature of mainstream image-processing platforms such as SciKit or OpenCV. As a result, even if texture-annotation protocols could be standardized for describing pattern-symmetry and persistent-homology parameters, it would be difficult to actually calculate the proper values for such annotations as part of a generic image-processing environment.

Given these sorts of considerations, we have devoted some attention to image analysis in particular, with the idea that certain useful forms of annotations require an underlying Computer Vision platform (in contrast to text annotations, for instance, which can be manually inserted while documents are being authored in the first place; Natural Language Processing is beneficial but not strictly necessary). In particular, we implemented a novel image format whose goal is to efficiently organize pixel-memory, reducing the number of operations needed to accomplish certain pipeline tasks. Tnese include computing local color histograms; creating thumbnail displays; mapping photos to graphics scenes for interactive segmentation; annotating Regions of Interest; and formulating local color models which employ extra channels — so as to disambiguate color-points which are visibly similar (or identical) but express discordant relations to their nearby context (e.g., a light brown/orange color should have a different interpretation within the texture of a leaf in early autumn and the texture of tree bark with sporadic highlights from the sun). In contrast to almost all raster image formats (except for "tiled" **TIFF**) — and also to conventions for encoding pictures in generic scientific media such as **HDF5** — our specific layout does not store pixels through two-dimensional arrays but rather positions them within each file based on geometric centrality (pixels closer to the edge are later in the file) such that subimages around the center can be obtained simply by truncating files as loaded into memory. This also makes certain potentially useful computations quicker to obtain than with raster matrices: in effect, some metadata is precomputed and stored (or implicitly encoded) inside the image, in conjunction with how the pixel-layout is partitioned.

Our long-term goal is to standardize an **API** for working with this specific image-format — by analogy to **H5IM** (the **HDF5** Image Module) — and in general to deploy image-processing code as analytic tools associated with this format, which would thereby serve as one specific multimedia file convention belonging to a general protocol for multimedia cross-referencing, alongside **GUI** and data-integration tools for

GIS, videos, and so forth.

Software, Cloud, and GUI Modules

As intimated in the preceding section, specific C++ modules may be published via DaSemio for files/resources of these different multimedia types, such that each parser/viewer would recognize annotations and cross-references and could interoperate with other modules based on cross-citation or "microcitation" links (cf., again, geotagging video frames). Our compiler and VM infrastructure is threaded through this overall ecosystem insofar as each multimedia viewer/processor could link against the VM as an embedded component, thereby utilizing it for loading metadata, fine-tuning application and GUI state, responding to user interaction, interoperating with host applications, and cross-module message passing.

In short, for any given multimedia genre (cartography, videos, etc.) DaSemio seeks to provide — or, in the case of narrower domain-specific formats, at least offer an infrastructure to help implement — reusable code libraries that could be dropped in to C++ applications (in particular). To the degree that such content could be displayed through semi-autonomous windows or subwindows, the relevant libraries are designed so that developers maintaining the host-application code base would not need expert knowledge of the media type itself, but instead would code the necessary connection-points between host and extension modules using generic programming conventions, such as Qt signals/slots or "reactive" Model/View/Controller objects.

Utilizing semi-autonomous GUI modules aids those who curate data sets because those modules help standardize application-level capabilities vis-à-vis the relevant data format(s). Data sets and their concomitant computer code can be designed for a specific GUI module rather than for a multiplicity of separate applications. Module-based standardization is more straightforward for media types shared by multiple domains (genres like video, GIS, etc.) as compared to more-specialized uses (such as 3D molcular visualization) where there may only be a small number of separate applications capable of displaying the relevant assets. In some cases, nonetheless, code from individual applications might actually be refactored into code libraries available for other software; but even when applications cannot be mutually embedded a consistent message-passing protocol permits shared functionality to a similar degree. Special-purpose data structures/formats, such as molecule models (referring back to the IQmol example) should therefore be considered a form of multimedia content, accessed via techniques similar to those apropos of more generic media types — 3D simulations of a chemical compound/formula are more specific than just a 3D graphic (because of additional visual and analytic parameters; e.g., ones related to atomic bonds), but this just means that molecular sketch-scenes are a more graular multimedia genre. In DaSemio, supporting domain- and discipline-specific formats may be achieved by embedding a common VM module in multiple applications for networking.

Cloud/Native and Peer-to-Peer Networks

Analogous principles apply to networking on a larger scale (than message-passing confined to a single computer, for instance). By design, DaSemio will incorporate cloud-oriented modules governed by similar philosophies as the other components. In particular, DaSemio cloud components can be shared as C++ source files and run via multiple options, including as local programs or within Docker containers as well as conventional HTTP servers. This allows for localized testing and prototypes. More to the point, DaSemio HTTP modules conventionally assume that important parts of their code base may be shared by client applications, blurring the distinction between "client" and "server" endpoints. We call this a "Peer-Hybrid Cloud/Native" (PHCN) application model, characterized by a level of shared procedural capabilities between clients and servers or among clients in decentralized networks — greater procedural alignment than is typically assumed for client/server architectures based on static (including REST) APIs. PHCN assumes that code implementing web sites/applications or services can be packaged and shared in a portable manner. To reiterate, we assume a general pattern wherein clients and servers share code libraries for the overlapping parts of their business logic; such libraries may then be subject to refinement and evolution across both server-side and client-side participants. Creating localized server-instances is one way to test and prototype libraries along these lines.

Simultaneously, PHCN services are optimized to be accessed from self-contained, desktop-style software; in the case of servers hosting web sites, clients would (at least as one option) employ dedicated windows which render web pages essentially as minitature browsers, via technologies such as **QWebEngine** — which functions as an instance of the Chrome browser local to a single (self-contained) application. When embedded in this manner, web sites/services potentially benefit from more sophisticated graphics, visualization and multimedia capabilities, exposed through the host application, than are possible through conventional web browsers — which share a more limited feature-set, simply by virtue of being constructed for traditional web pages rather than native-oriented content. In effect, web-site embedding represents one strategy for publishing variegated but interrelated multimedia content; a project's resources could be hosted and organized via contentional web techniques, but client-application features may be requested from within individual web pages during sessions while they are viewed from embedded browser engines (e.g., by JavaScript callbacks to C++ handlers).

In addition to superior user experience, web-site embedding allows host applications to unify several isolated web sites which are logically (but not logistically) interrelated. This is particularly appropriate for governmental or environmental services, that tend to be artificially separated by jurisdictional boundaries. For example, a researcher or governmental official interested in the environmental impact of

municipal decisions — such as sponsoring public/private initiatives or zoning and land-use classifications — will find relevant data scattered across multiple websites partitioned by geographic boundaries. In concrete terms, consider the New York metro area: portals such as NYCPlanning's ZoLa system, which provides zoning and property info for the five boroughs, are blank across the river in New Jersey; while analogous resources such as New Jersey's Department of Environmental Protection (NJ-DEP) interactive maps contain no information back in New York. Most environmental risk factors are shared between the states of New York and New Jersey — hurricanes, flooding via the Hudson River or Atlantic Ocean, poor air quality, and so on — and likewise for trends in (say) housing, transit, sociodemographics, etc. Unfortunately, however, because state agencies on opposite sides of the Hudson River operate separately from one another (and answer to disparate governments — Trenton vs. Albany) there is little incentive for New Jersey and New York State digital resources/platforms to be synchronized or interoperate. One way to alleviate this problem is to build special-purpose applications that toggle between web sites hosted by the two states respectively, which can include acquiring and synchronizing data from both sides.

For standalone applications hosting multiple embedded web sites, a canonical use-case would indeed be user actions on GIS map displays: almost all such events should be interpreted through the GIS coordinates where an action occurs, which are a product of any digital map's current display state (panning and zoom levels — the latter specifying the degree of details and magnification for GIS elements and the former measuring how users have dragged the map north, aourh, east, or west, determining the latitude and longitude of the map's current center — plus which data layers are currently visible). Unfortunately, there is no standard protocol for obtaining this information once GIS web displays are embedded in desktop software. The host application knows window coordinates where events take place, but programmers lack a single (or guaranteed) algorithm to convert those coordinates to GIS locations (or street addresses). Even where a host application can infer this data indirectly from an embedded web site (e.g., via URL patterns) the techniques for doing so will typically be idiosyncratic for each embedded resource, so each site needs manual coding for the relevant event-handlers.

Adopting a common embedding protocol, by contrast, would enable embedded web sites to notify host applications of required event metadata via a shared model so that new peer web sites could be added with minimal extra programming. With such techniques, geographically related sites could be stitched together almost seamlessly, so one might for instance transition from land-use queries on New York territory to New Jersey via a front-end interface (in terms of context menus, secondary windows, data-layer styling, etc.) which is largely the same on either side of the Hudson River. Such a common interface style could then be extended beyond digital maps proper to other assets that may be used in conjunction with GIS services — for example, linking place-names as GIS locations (with implicit latitude and longitude points/boundaries) to text-segments in HTML or PDF files (GIS elements, such as names of counties and cities or towns/boroughs, becoming treated as Named Entities with a specific kind of associated metadata, viz., geographic coordinates and boundary-shapes).

Data Sets, Queries, and Database Architecture

Technology focused on data *sets* is, for obvious reasons, somewhat different from *database* engines and their components: data sets, in general, are characterstically static and only change from one version to another, whereas databases are typically dynamic and subject to modification in real time. Of course, a data set might be derived from a database, capturing its state at a given moment in time. More generally, database tools are a common instrument for curating data sets; the latter in effect become statically captured — and potentially refined, cleaned, or normalized — views onto the former. Publishing a data set implies that the underlying data-acquisition process (obtaining measurements, performing tests/experiments, and so forth) has been completed, or at least reached a periodic milestone, so that the set marshals information reflecting an extended but past-tense acquisition period; whereas a database holds transient content that is liable to change at any time (and is still "live" for the immediate future).

Despite these differences, databases and sets are both structured information archives and have similar technological requirements in many respects, including concerns about structural models and about access — how specific information (more granular than what is available in the entirety) might be obtained, selected, filtered, and analyzed. It is a reasonable goal, for instance, to deploy query languages that can work against both static data sets and dynamic database engines. Likewise, software components whose goal is to load data sets into host applications might be most useful if their information sources could also be configured as database instances (with connections to databasees serving as replacements for loadable data set files) and vice-versa.

Given these considerations, DaSemio formulates a generic model of "views" on an information source (agnostic as to whether that source is more like a data set, database, or something else or in-between) to support query and bridge code that adapts variegated sources to the common view/access protocol. The basic idea is a definition of data-source "sites," which are specific locations from which one can read/consume a computational value. For instance, in a relational table (or even a spreadsheet/CSV file) one site could be a row, column, or a row/column intersection. In a NoSQL property graph, conversely, a site might be an individual node, or a single key/value pair stored as one property *of* an individual node. In general, according to this convention, each data-source site would be representable via a unique label, built up from smaller parts in consort with the overarching architecture: record-then-field, say, for SQL, or node-then-attribute, for property graphs. It is assumed that software "adapting" data sources would be able to convert an unambiguous site-description into a single value or collection thereof (decoded or deserialized if necessary into an application-level type instance) as well as to maintain a

form of “cursor” and navigate the data source by switching (or “hopping”) from one site to another.

When a data set is prepared for publication, accordingly, such a site-model may guide how its contents are organized and serialized. The proper formats and structural layouts should be chosen to enable adapters within client applications to query and traverse within the information provided, according to standardized schemas for site labeling and navigation.

Some References

Although we have not produced a comprehensive review of this system in its entirety, specific parts therein have been presented/analyzed in a handful of published articles and chapters. Given its focus on manipulating data sets derived from a scientific/research environment, an emphasis of the compiler/VM infrastructure is to support design-by-contract insofar as contractual specification can clarify and document the scientific protocols through which data sets are acquired/generated. We discuss techniques for implementing design-by-contract within compiler stacks in articles such as chapter 3 in “Advances in Ubiquitous Computing: Cyber-Physical Systems, Smart Cities and Ecological Monitoring” and chapter 6 of the “Innovative Data Integration” text mentioned earlier. Our new image-representation format was outlined in Chapter 22 of “AI, IoT, Big Data and Cloud Computing for Industry 4.0” (Springer, 2023). That 2023 book’s chapters 20-21, and the Covid/Oncology book’s chapters 5 and 9, analyzed type systems via constructions relevant for compiler theory and, as such, present some of the technical background behind certain atypical features of our compiler framework (the central idea being to semantically associate types with contract-style certifications of execution vis-a-vis data constructors, which yields a noticeably different foundation for type theory than the predominant set-, model-, or Category-theoretic paradigms). Also, Chapter 4 of the Covid/cancer book includes the aforementioned analysis of along with examination of different data-set, digital-modeling, and extensibility protocols for various examples of scientific software (to clarify the overall problem-space of using native-compiled scientific software as tools for reading and reusing data sets); and chapters 6 and 9 there outlined a a parsing theory reflecting both computer and natural languages, addressing early stages of compiler pipelines. Our text-encoding system inspired by AI2’s “call to action” was featured most prominently in supplemental material published in conjunction with an article titled “Age-Related Hearing Loss, Speech Understanding and Cognitive Technologies” from the *International Journal of Speech Technology* (cited as Lehmann *et. al.*, 2021) that leveraged a cross-referencing system synchronizing text documents (compiled within the data set) and samples from cognitive-linguistic/pragmatics corpora.

Video Links

For more information, please see our software demo vidoes: